SQL PLAYGROUND · SETUP GUIDE

Deploy on a fresh VPS

Ubuntu, MySQL, nginx and ufw, behind Cloudflare, with no Docker: from a blank server to https://your-domain in about 40 minutes.

11
copy-and-paste steps

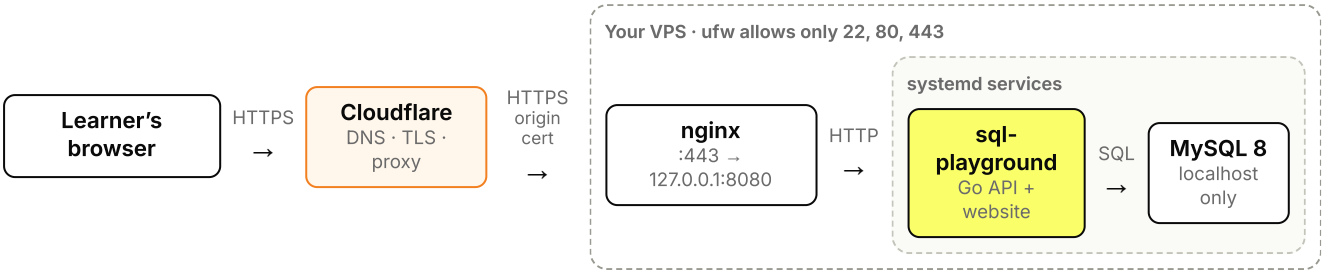
3
open ports: SSH, HTTP, HTTPS

0
database ports exposed

OVERVIEW

What you'll end up with

A fresh Ubuntu server running SQL Playground at **https://sql.example.com**, behind Cloudflare, with only SSH and HTTPS open to the internet. No Docker: MySQL and the app run as ordinary system services. The database is never exposed, and every learner gets a private copy of the data inside MySQL.



Before you start

- [ ] A VPS with **Ubuntu 24.04 LTS** (22.04 also works), **8 GB RAM**, at least **20 GB free disk**, and SSH access as root (or a sudo user).
- [ ] Your domain on **Cloudflare**, with an A record pointing at the server. You'll check its settings in step 7.
- [ ] Access to the GitHub repository **darshan260802/sql-playground**.
- [ ] About **40 minutes**. Everything is copy and paste; run commands one block at a time.

Values used in this guide

In this guide	Replace with
`sql.example.com`	the hostname you'll use for the playground, e.g. *sql.yourdomain.com* (or just *yourdomain.com*)
`203.0.113.10`	your server's public IPv4 address
`darshan`	the Linux user you create in step 1 (any name works)

Placeholders are highlighted inside commands. Lines starting with `#` are comments; pasting them into the terminal is harmless.

1 Log in and create your admin user

Day-to-day work happens as a normal user with sudo, not as root.

From your own computer (Terminal on macOS/Linux, PowerShell on Windows), log in as root:

```
ssh root@203.0.113.10
```

Create the user, give it sudo, and copy root's SSH keys to it (if you log in with a key):

```
adduser darshan
usermod -aG sudo darshan
rsync --archive --chown=darshan:darshan ~/.ssh /home/darshan
```

Keep this root session open. In a **second** terminal on your computer, log in as the new user:

```
ssh darshan@203.0.113.10
```

> **TIP** If you log in with a password and have no SSH key yet, create one on your computer with `ssh-keygen -t ed25519` and copy it to the server with `ssh-copy-id darshan@203.0.113.10`. Step 10 turns off password logins, so set up a key before you get there.

From here on, run everything as **darshan**. Commands that need root start with `sudo`.

---

2 Update the system

Latest security fixes, automatic updates, and a small swap file as a safety net.

```
sudo apt update && sudo apt -y full-upgrade
sudo apt -y install git curl ca-certificates unattended-upgrades
sudo timedatectl set-timezone UTC
# choose "Yes" to install security updates automatically
sudo dpkg-reconfigure -plow unattended-upgrades
```

Swap (recommended): 2 GB of swap keeps the first build from running out of memory.

```
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

> **NOTE** If the upgrade asks whether to keep a modified configuration file (for example sshd_config), keep the version currently installed. If it installed a new kernel, reboot with `sudo reboot`, wait a minute, and log in again as darshan.

---

3 Turn on the firewall (ufw)

Block everything incoming except SSH. Web ports are opened in step 4.

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH
sudo ufw enable
sudo ufw status verbose
```

> **IMPORTANT** Always allow SSH **before** `ufw enable`, or you lock yourself out. If your SSH runs on a custom port, also run `sudo ufw allow 2222/tcp` (with your port).

> **NOTE** Some providers (AWS, Oracle, Hetzner Cloud, ...) also have a firewall in their web panel. If you use one, allow TCP 22, 80 and 443 there too.

4 Install nginx

nginx receives HTTPS traffic from Cloudflare and hands it to the app.

```
sudo apt -y install nginx
sudo ufw allow 'Nginx Full'
# the default "Welcome to nginx" site isn't needed
sudo rm /etc/nginx/sites-enabled/default
sudo systemctl reload nginx
```

CHECK `sudo ufw status` now lists **OpenSSH** and **Nginx Full** (ports 80 and 443).

5 Get the code

Clone the repository into /opt/sql-playground.

```
sudo mkdir -p /opt/sql-playground
sudo chown darshan:darshan /opt/sql-playground
git clone https://github.com/darshan260802/sql-playground.git /opt/sql-playground
```

If the repository is private, the clone asks for credentials. Use a read-only deploy key instead:

```
ssh-keygen -t ed25519 -C "vps deploy key" -f ~/.ssh/github_deploy -N ""
cat ~/.ssh/github_deploy.pub
```

On GitHub open the repository → **Settings** → **Deploy keys** → **Add deploy key**, paste the key (leave "Allow write access" off), then:

```
printf 'Host github.com\n  IdentityFile ~/.ssh/github_deploy\n  IdentitiesOnly yes\n' \
  >> ~/.ssh/config
chmod 600 ~/.ssh/config
git clone git@github.com:darshan260802/sql-playground.git /opt/sql-playground
```

6 Install the playground

One script installs MySQL, builds the app and starts it as a service. You'll run it again for every update.

```
cd /opt/sql-playground
sudo ./deploy/install.sh
```

It takes 2–5 minutes the first time and prints each stage as it goes:

- installs **MySQL 8** from Ubuntu's packages and adds the playground's settings (`deploy/mysql/playground.cnf`);
- creates a MySQL account for the app with a random password, saved in `/etc/sql-playground/config.env`;
- downloads **Go** and **Node.js** into `/usr/local/lib/sql-playground/toolchain` and builds the app. They're only used for building and don't change anything else on the server;
- installs and starts the **sql-playground** service, the `sqlpg-admin` command and a nightly backup;
- waits until the app answers. The first start generates the dataset, about 30 seconds.

CHECK It ends with **"SQL Playground is running on http://127.0.0.1:8080"**. If it stops with an error instead, the message says what went wrong; fix it and run the same command again (it's always safe to re-run).

```
curl -s http://127.0.0.1:8080/api/health
systemctl status sql-playground --no-pager | head -n 3
```

CHECK The first command prints `{"status":"ok"}`; the second shows **active (running)**. MySQL and the app start automatically after a reboot.

The settings live in `/etc/sql-playground/config.env`. The defaults are right for this guide:

Setting	Default	Meaning
`SQLPG_LISTEN`	127.0.0.1:8080	where nginx reaches the app (localhost only)
`SQLPG_COOKIE_SECURE`	true	keep true: the site is only used over HTTPS
`SQLPG_MAX_USERS`	20	sign-ups close at this many accounts
`SQLPG_USER_QUOTA_MB`	300	storage limit per learner database

To change one: `sudo nano /etc/sql-playground/config.env`, then `sudo systemctl restart sql-playground`.

NOTE **Port 8080 already taken** by another project? Install with `sudo ./deploy/install.sh --port 8081` and use that port in step 8. **MySQL already used by other projects?** The playground's MySQL settings are server-wide (no binary log, no LOAD DATA, UTC time zone); read `deploy/mysql/playground.cnf` first.

7 Configure Cloudflare

DNS through Cloudflare's proxy, end-to-end encryption, and a free 15-year origin certificate.

In the Cloudflare dashboard, open your domain. Menu names can move slightly over time; the settings stay the same.

7a · DNS record

DNS → Records: make sure there is an **A** record for your hostname (Name `sql` for sql.example.com, or `@` for the bare domain) pointing to `203.0.113.10`, with **Proxy status: Proxied** (orange cloud). If the server has IPv6, add an **AAAA** record the same way.

7b · Encryption mode

SSL/TLS → Overview: set the mode to **Full (strict)**.

7c · Edge certificates

SSL/TLS → Edge Certificates: turn on **Always Use HTTPS** and **Automatic HTTPS Rewrites**, and set **Minimum TLS Version** to **1.2**.

7d · Origin certificate (encrypts Cloudflare → your server)

SSL/TLS → Origin Server → Create Certificate. Keep "Generate private key and CSR with Cloudflare", key type **RSA (2048)**, list your hostname (e.g. `sql.example.com`; adding `*.yourdomain.com` also works),

validity **15 years**, then **Create**. Leave the page open: it shows two text blocks.

```
sudo mkdir -p /etc/ssl/cloudflare
# paste the "Origin Certificate" block, save, exit
sudo nano /etc/ssl/cloudflare/sql-playground.pem
# paste the "Private Key" block, save, exit
sudo nano /etc/ssl/cloudflare/sql-playground.key
sudo chmod 600 /etc/ssl/cloudflare/sql-playground.key
```

IMPORTANT Cloudflare shows the private key **only once**. Paste both blocks completely, including the BEGIN and END lines. The origin certificate is trusted by Cloudflare only, so keep the DNS record **Proxied**.

7e · Recommended extras

- **Speed → Optimization → Rocket Loader:** Off. It rewrites JavaScript, which can leave a blank page.
- **Caching → Configuration → Browser Cache TTL:** "Respect Existing Headers" (the app already sends the right cache headers).
- **Security → Bots → Bot Fight Mode:** if it's on and queries fail with "Can't reach the server", turn it off.

---

8 Connect nginx to the app

HTTPS with the origin certificate, plus real visitor IPs for the app's rate limits.

Install the site config from the repository and put your hostname in it:

```
# your hostname
DOMAIN=sql.example.com
sudo cp /opt/sql-playground/deploy/nginx/sql-playground-cloudflare.conf \
  /etc/nginx/sites-available/sql-playground.conf
sudo sed -i "s/sql\.example\.com/$DOMAIN/g" /etc/nginx/sites-available/sql-playground.conf
sudo ln -s /etc/nginx/sites-available/sql-playground.conf /etc/nginx/sites-enabled/
grep server_name /etc/nginx/sites-available/sql-playground.conf
```

CHECK The last command shows your hostname twice. If you installed on another port (step 6), also edit the `server 127.0.0.1:8080;` line in that file.

Behind Cloudflare, every request appears to come from a Cloudflare server. This script makes nginx trust Cloudflare's published IP ranges, so the app sees each visitor's real IP. It also checks the nginx configuration and reloads it:

```
sudo /opt/sql-playground/deploy/scripts/cloudflare-realip.sh
```

CHECK The script ends with "nginx trusts NN Cloudflare ranges". If it stops with an error, see Troubleshooting at the end of this guide.

If `nginx -t` reports *"Address family not supported by protocol"*, the server has no IPv6. Remove the IPv6 listen lines and run the script again:

```
sudo sed -i '/listen \[::\]/d' /etc/nginx/sites-available/sql-playground.conf
sudo /opt/sql-playground/deploy/scripts/cloudflare-realip.sh
```

9 Test everything

Check the whole path, from the browser to the database.

1. Open **https://sql.example.com**. The playground loads with a padlock in the address bar.
2. Press **Run** on the welcome query. Results appear within a couple of seconds.
3. Click **Get started** and create your account. A yellow bar shows your private copy being prepared (about 10 seconds), then **My database** becomes active.
4. On **My database**, run `UPDATE products SET unit_price = unit_price * 1.1 WHERE category_id = 41;` and see “rows affected”.
5. Open **Learn**, solve the first exercise, and press **Check**.

On the server:

```
curl -sI https://sql.example.com | grep -iE '^(HTTP|server)'
sudo tail -n 3 /var/log/nginx/access.log
sudo sqlpg-admin list-users
```

CHECK You see `HTTP/2 200` and `server: cloudflare`. The access log starts with **your own** public IP, not a Cloudflare address (104.x, 172.64–71.x, …), which confirms step 8. The last command lists the account you just created.

10 Harden the server

Recommended: lock the web ports to Cloudflare, turn off password logins, add fail2ban.

10a · Only Cloudflare may reach ports 80 and 443

This stops anyone from bypassing Cloudflare by connecting to the server IP directly:

```
sudo /opt/sql-playground/deploy/scripts/cloudflare-realip.sh --ufw
sudo ufw status numbered
```

NOTE If you ever switch the DNS record to “DNS only” (grey cloud), open the ports again with `sudo ufw allow 'Nginx Full'`.

10b · SSH: keys only, no root login

Do this only after logging in as darshan **with a key** works:

```
sudo tee /etc/ssh/sshd_config.d/99-hardening.conf > /dev/null <<'EOF'
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
EOF
sudo sshd -t && sudo systemctl restart ssh
```

IMPORTANT Keep your current session open and test a **new** SSH login before closing it. If the new login fails, undo the change by deleting that file from the open session.

10c · fail2ban and Cloudflare range updates

```
sudo apt -y install fail2ban
sudo crontab -e
```

Add this line at the bottom (a monthly refresh of Cloudflare's IP ranges), then save:

```
23 4 1 * * /opt/sql-playground/deploy/scripts/cloudflare-realip.sh >/dev/null 2>&1
```

Backups are already scheduled: the installer added a nightly backup at 03:17 to `/var/backups/sql-playground/` (the last 14 are kept). They contain accounts, saved queries, history and lesson progress. Learners' database copies aren't included; the app rebuilds them automatically after a restore. The backup folder is readable by root only, so to keep a copy off the server, first copy the files to your home folder, then download them from your computer:

```
# on the server
sudo sh -c 'cp /var/backups/sql-playground/app-*.sql.gz ~darshan/ &&
  chown darshan: ~darshan/app-*.sql.gz'
# on your computer
scp 'darshan@203.0.113.10:app-*.sql.gz' .
```

11 Everyday operations

The commands you'll actually use.

Task	Command
Status / logs	`systemctl status sql-playground` `journalctl -u sql-playground -f`
List accounts and storage	`sudo sqlpg-admin list-users`
Reset a forgotten password (prints a new one)	`sudo sqlpg-admin reset-password EMAIL`
Restore a learner's data to the original	`sudo sqlpg-admin reset-db EMAIL`
Delete an account and its database	`sudo sqlpg-admin delete-user EMAIL`
Update to the latest code	`cd /opt/sql-playground && git pull` `sudo ./deploy/install.sh`
Change a setting	`sudo nano /etc/sql-playground/config.env` `sudo systemctl restart sql-playground`
Restart / stop	`sudo systemctl restart sql-playground` `sudo systemctl stop sql-playground`
Disk usage	`df -h /` `sudo du -sh /var/lib/mysql`
Back up now	`sudo /usr/local/lib/sql-playground/backup.sh`
Find out why the site doesn't load	`sudo /opt/sql-playground/deploy/scripts/doctor.sh sql.example.com`

IMPORTANT Never delete `/var/lib/mysql` or run `apt purge mysql-server`: MySQL's data folder holds every account and database.

Restore a backup (same server or a new one)

Set up the new server with steps 1–8 first, copy the backup file to it, then:

```
sudo gunzip -c app-2026-01-31.sql.gz | sudo mysql
sudo systemctl restart sql-playground
```

Accounts, saved queries and progress come back, and each learner's database is rebuilt in the background (about 10 seconds each).

What to expect

MySQL uses about 1.2 GB of RAM and the app about 30 MB. Disk: about 1.1 GB for MySQL and the core data, plus 148 MB per learner (up to the 300 MB limit), so 20 learners need roughly 3–6 GB. The build tools and caches take another 650 MB.

? Troubleshooting

Cloudflare error pages show a number. Start there.

Symptom	Likely cause	Fix
Blank page, and the tab keeps loading	part of the website doesn't reach the browser	run `sudo /opt/sql-playground/deploy/scripts/doctor.sh sql.example.com` : it tests app → nginx → Cloudflare and says where it breaks. Turn Rocket Loader off (step 7e)
Cloudflare **521** "Web server is down"	nginx stopped, or a firewall blocks 443	`sudo systemctl status nginx` ; `sudo ufw status` must allow 443 (Nginx Full or Cloudflare rules); check the provider's firewall
Cloudflare **522** "Connection timed out"	wrong IP in the DNS record, or a provider firewall drops traffic	compare the A record with the server's IP; open 80/443 in the provider panel
Cloudflare **525** "SSL handshake failed"	nginx isn't serving HTTPS on 443	`sudo nginx -t` ; check the site is in sites-enabled and the certificate paths exist
Cloudflare **526** "Invalid SSL certificate"	Full (strict) but the certificate isn't the origin certificate, or it doesn't list this hostname	create a new origin certificate that includes the hostname (step 7d) and replace both files
"Too many redirects"	SSL/TLS mode is "Flexible"	set **Full (strict)** (step 7b)
nginx **502 Bad Gateway**	the app isn't running, or the port doesn't match	`systemctl status sql-playground` ; `journalctl -u sql-playground -n 50` ; SQLPG_LISTEN must match the upstream port in the nginx file
`nginx -t` : "Address family not supported by protocol"	the server has no IPv6	remove the `listen [::]` lines (step 8)
Signed out right after signing in	the site was opened over http://	always use https:// (turn on Always Use HTTPS, step 7c); keep SQLPG_COOKIE_SECURE=true
Everyone gets "You're running queries very quickly"	real visitor IPs aren't set up, so all traffic looks like one Cloudflare IP	run `sudo /opt/sql-playground/deploy/scripts/cloudflare-realip.sh`
Queries fail with "Can't reach the server"	Bot Fight Mode or a WAF rule challenges API calls	turn off Bot Fight Mode, or skip security for `/api/` paths
"This playground is full"	SQLPG_MAX_USERS reached	raise it in `/etc/sql-playground/config.env` and restart the service, or delete unused accounts
install.sh: "port 8080 is already used"	another program listens there	`sudo ./deploy/install.sh --port 8081` (first install) or change SQLPG_LISTEN; update the nginx upstream
install.sh: "can't log in to MySQL as root"	MySQL's root user has a password (not Ubuntu's default)	put it in `/root/.my.cnf` under `[client]` as `password=…` , then run install.sh again
install.sh: "the app keeps stopping"	the app can't start; the log above the message says why	`sudo systemctl status mysql` ; `journalctl -u sql-playground -n 50` ; if MySQL ran out of memory, lower `innodb_buffer_pool_size` in deploy/mysql/playground.cnf and re-run
Build stops with "Killed"	not enough memory while building	add swap (step 2) and run install.sh again

A Files at a glance

Where everything lives on the server.

Path	What it is
`/opt/sql-playground`	the repository (code, deploy files); run `git pull` here to update
`/etc/sql-playground/config.env`	settings and the app's MySQL password (root-only; keep private)
`/etc/systemd/system/sql-playground.service`	the service; runs as the unprivileged user `sqlpg`
`/usr/local/lib/sql-playground/`	the installed app (`bin/sqlpg` , `web/`), backup script, build tools and caches
`/usr/local/bin/sqlpg-admin`	admin commands (step 11)
`/etc/mysql/mysql.conf.d/zz-sql-playground.cnf`	MySQL tuning (buffer pool, safety settings)
`/var/lib/mysql`	all MySQL data (core dataset, learner copies, app tables)
`/etc/cron.d/sql-playground`	the nightly backup
`/var/backups/sql-playground/`	backups of accounts and progress (last 14)
`/etc/nginx/sites-available/sql-playground.conf`	the nginx site (HTTPS + proxy to the app)
`/etc/nginx/conf.d/cloudflare-realip.conf`	generated list of trusted Cloudflare ranges
`/etc/ssl/cloudflare/`	the Cloudflare origin certificate and private key

The order, in one glance

1. user + SSH key
2. updates + swap
3. ufw: SSH
4. nginx + ufw: web
5. clone
6. install.sh
7. Cloudflare: DNS, Full (strict), origin cert
8. nginx site + real IPs
9. test
10. harden
11. operations